Lab 4 — Agents: Plan Mode, Local Agent, and Worktrees
After Module 4 — Copilot Agents
You will build Version 2 with three different agent surfaces and feel the difference between them.
Confirm git status is clean and everything is pushed.
Before you start: the starter repository already contains .github/agents/planner.agent.md. You will use it in Lab 7.

Part A — Prepare
Step 1 — Move the project to Version 2
Your custom instructions currently forbid a database. That was correct for Version 1 and is now wrong. Open .github/copilot-instructions.md.
Under Project, change the version list so Version 2 is marked as current:

```
- Version 1 (complete): in-memory storage, no database
- Version 2 (current): SQLite via SQLAlchemy with Alembic migrations
- Version 3: Category field added to todos
```

Under Do NOT, change the database line to:

```
- Add a Category field. That is Version 3.
```

Commit before going further:

```
git add .
git commit -m "Move project to Version 2"
git push
```

Part B — Plan mode
Step 2 — Switch to Plan mode
Set the mode picker to Plan and the model to Claude Sonnet 5. Plan mode designs before it builds. It will not write code until you approve.

Step 3 — Ask for the Version 2 plan

```
Design the implementation of Version 2 for this project.

Goal: replace the in-memory todo store with a SQLite database, so that
todos survive a server restart.

Requirements:
 - SQLAlchemy for the data layer.
 - Alembic for migrations. The initial migration must create the
   todos table.
 - Only the repository layer may change. Routers, services, Pydantic
   schemas, and the entire frontend must remain untouched.
 - The database file lives at backend/todos.db and is gitignored.
 - Field names and API behaviour do not change.

Read #file:docs/TRD.md section 7, which states exactly which files
should change when SQLite is introduced.

For the plan, list:
 1. Every file to be created, with its purpose.
 2. Every file to be modified, and what changes in it.
 3. The order of steps.
 4. The commands that must be run and when.
```

```
5. Anything you are uncertain about.

Do not write any code.
```

Read the plan and check exactly two things:

- Does it modify any file outside backend/repository/?
- Does it list alembic init, then alembic revision, then alembic upgrade head, in that order?

If either check fails, push back in the same conversation:

```
Your plan modifies files outside the repository layer. The TRD requires
that only the repository changes. Revise the plan to respect that
boundary, or explain why it is not possible.
```

Part C — Agent mode in the Chat window

Step 4 — Hand the plan to the local agent

At the end of the plan, click Start Implementation. Choose the default agent. The plan and the whole conversation carry over, and the mode picker switches to Agent.

We run this locally, in place, because it needs the terminal, needs to run alembic upgrade head, and needs the server restarted to verify. That is what local agent mode is for.

Step 5 — Watch the loop

As the agent works, watch the chat panel. You will see it read files, propose terminal commands for approval, run them, read the output, and correct itself.

Approve each command after reading it. Do not blind-approve.

If the migration fails, do not fix it yourself. Let the agent read the error and retry. That self-correction loop is what you are here to observe.

Step 6 — Test persistence and commit

Restart the backend:

```
cd backend
uvicorn main:app --reload
```

Add three todos in the browser. Stop the backend with Ctrl + C. Start it again. Refresh the browser. The todos are still there. Version 2 works.

```
git add .
git commit -m "Version 2: SQLite persistence via SQLAlchemy and Alembic"
git push
```

Part D — Agent Window with the Copilot harness and a worktree

The next feature is independent of the database work. You will run it in a separate process, in an isolated copy of the repository, while you keep working.

Step 7 — Start a Copilot session in a new worktree

Press Ctrl + comma and search for chat.agentHost.enabled. Confirm it is on. If your organisation has disabled it, read Steps 7 to 9 and move to the discussion.

- In Copilot Chat, open the Session Target dropdown.
- Select Copilot.
- Tick New Worktree.
- Note the permission level shown — worktree isolation fixes it at Bypass Approvals, because the agent works on an isolated copy and cannot damage your working tree.

The session opens in the Agent Window.

Step 8 — Give it an independent feature

```
Add filtering and sorting to the Todo application.
```

```
Backend:
  GET /todos should accept two optional query parameters:
    completed - true or false. Omitted means all todos.
    sort      - either "created_asc" or "created_desc".
                Default is created_desc.
  Filtering and sorting happen in the repository layer using SQLAlchemy,
  not in Python after fetching all rows.

Frontend:
  Add a filter control with three options: All, Active, Completed.
  Add a sort control with two options: Newest first, Oldest first.
  Both call the API with the appropriate query parameters. Do not filter
  or sort in the browser.

Also add a "Clear all completed" action:
  DELETE /todos/completed removes every completed todo and returns the
  number deleted. The service method is named clear_completed. Only
  completed todos are deleted.
  Add a matching button to the frontend.

Follow the layer rules in #file:.github/copilot-instructions.md.
Do not change the Todo model or add any field.
Run the backend and confirm the endpoint responds before you finish.
```

Do not sit and watch. While the session runs, go back to the editor and open docs/TRD.md, and add a short note recording that Version 2 is complete.
Notice that the editor is not blocked. This is the difference between the Agent Window on the Agent Host and the local agent from Part C.

Step 9 — Review and merge the worktree

- In the Agent Window, review the diff. The agent auto-committed each turn in the worktree.
- Confirm the SQLAlchemy query does the filtering, not a Python list comprehension after the fact.
- Merge the worktree branch into your working branch.
- Run the application and test all six combinations of filter and sort.

```
git add .
git commit -m "Add filtering, sorting and clear completed"
git push
```

Step 10 — Compare the two surfaces

Answer this before you leave the lab: which of the two agent sessions blocked your editor, and which one would you use if you had to run three features at once?

What you should have at the end of this lab

```
TodoApp/
  backend/alembic/
  backend/alembic.ini
  backend/repository/   (now SQLAlchemy)
```

Version 2 persists to SQLite. Filtering, sorting and Clear all completed work. Todos survive a restart.